

FORGE

Framework for Orchestrated Requirements, Governance & Engineering

Un sistema di sviluppo software strutturato per OpenCode, Claude Code e Codex CLI

Cosa è FORGE?

FORGE è una metodologia completa per lo sviluppo software assistito da AI che combina:

- **Contesto progressivo strutturato** che previene decisioni inconsistenti
- **Processo adattivo** che calibra la cerimonia alla complessità
- **Conoscenza persistente** che sopravvive ai confini delle sessioni

Multi-piattaforma dalla v2.0 — OpenCode, Claude Code e Codex CLI — sintetizza il meglio di BMAD Method e Speckit

II Problema

5 Problemi Critici nello Sviluppo AI-Assisted

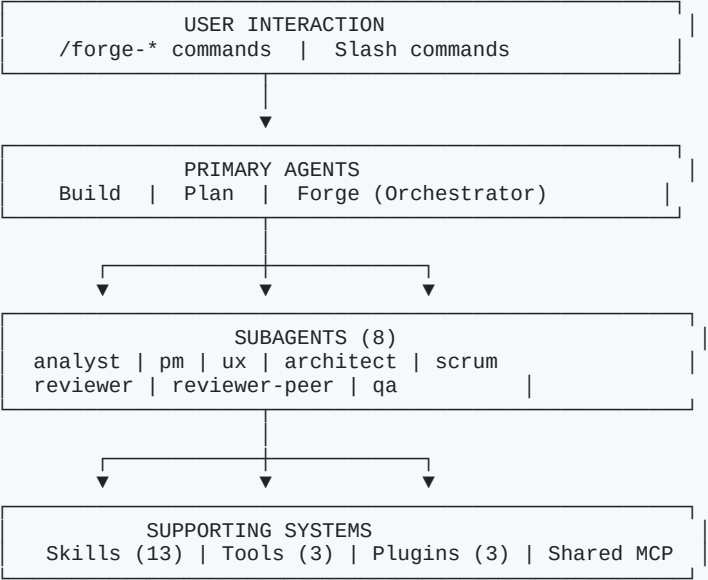
1. **Context Gap** - Agenti in sessioni diverse prendono decisioni contrastanti
2. **Ceremony Trap** - Troppo processo uccide la velocità, troppo poco la qualità
3. **Knowledge Evaporation** - Le decisioni scompaiono alla fine della sessione
4. **Consistency Entropy** - Team di 15+ sviluppatori producono codice inconsistente
5. **Illusion of Productivity** - Velocità senza direzione è spreco

La Soluzione FORGE

6 Principi Fondamentali

Principio	Descrizione
Progressive Context Engineering	Ogni fase produce un documento che diventa contesto per la fase successiva
Constitutional Governance	Principi immutabili governano ogni decisione
Adaptive Ceremony	Profondità del processo scala con la complessità
Adversarial Quality	Le review DEVONO trovare problemi
Persistent Knowledge	Decisioni e lezioni sopravvivono tra sessioni
Bidirectional Traceability	Requisiti → codice → test (e viceversa)

Architettura del Sistema



> v2.0: gli artifact canonici in ``.opencode/`` sono proiettati per piattaforma –
> ``.claude/`` per Claude Code, ``.codex/`` per Codex CLI – con un unico MCP server condiviso

v2.0: Multi-Piattaforma by Design

```
| Piattaforma | Vendor | Agent & Command | Config | Istruzioni |
|-----|-----|-----|-----|-----|
| **OpenCode** | OpenCode | `.opencode/{agents,commands,skills}/` | `opencode.json` | `AGENTS.md` |
| **Claude Code** | Anthropic | `.claude/{agents,commands,skills}/` + hooks | `.claude/settings.json` | `CLAUDE.md` |
| **Codex CLI** | OpenAI | `.codex/{agents,commands}/` + `.agents/skills/` | `.codex/config.toml` | `AGENTS.md` |
```

****Un installer auto-detecta il tuo runtime – stessa metodologia ovunque****

```
```bash
npx tsx install-forge.ts /path/to/project # auto-detect
npx tsx install-forge.ts /path/to/project --platform=claude-code
```

## Track: Quick

**Quando:** Piccola feature, 1-5 task, < 1 giorno

```
/forge-quick "Add forgot password feature with 1-hour expiry token"
```

### Workflow:

1. ✓ Quick Spec - Conversazione → tech-spec.md
2. ✓ Implement - Implementa task by task
3. ✓ Test - Genera unit test
4. ✓ Review - Adversarial self-review

**Output:** .forge/specs/NNN-name/tech-spec.md + codice + test

## Track: Feature

**Quando:** Feature media, 5-20 task, 1-5 giorni

```
/forge-specify "Add OAuth2 authentication with Google and GitHub"
/forge-clarify # Risolve ambiguità
/forge-ux # UX design: personas, wireframes, accessibilità
/forge-plan # Piano tecnico
/forge-analyze # Cross-valida spec vs plan
/forge-tasks # Task breakdown con dipendenze
/forge-implement # Implementa
/forge-review # Adversarial review (7 dimensioni, dual-model)
```

**Output:** spec.md, design-spec.md, user-journey.md, plan.md, tasks.md, ADR  
opzionali

# Fase UX: /forge-ux

**Quando:** Feature con interfaccia utente (web, mobile, design system)

/forge-ux "Login page with OAuth"

Artefatto	Contenuto	Focus
design-spec.md	Wireframe ASCII, componenti UI, specifiche interazione	 wireframe ·  design system ·  responsive
user-journey.md	Personas, scenari, flussi utente	 journeys ·  WCAG 2.1 AA

# Review: 7 Dimensioni di Qualità

## La review copre tutte e 7 le dimensioni:

Correctness	→ logica, edge case
Security	→ vulnerabilità, injection
Performance	→ query, memory leak
Maintainability	→ leggibilità, coupling
Constitution	→ aderenza ai principi
Test-Spec Coherence	→ test coprono i requisiti della spec
UX Quality	→ accessibilità, usabilità, design coerenza

## Esempi di issue UX:

- [HIGH] Bottone senza `aria-label` — non accessibile con screen reader
- [MEDIUM] Form senza focus management — UX degradata dopo submit
- [LOW] Contrasto colore 2.8:1 — sotto soglia WCAG AA (4.5:1)

## Track: Epic

**Quando:** Feature set complessa, 20-50+ task, 1-4 settimane

```
/forge-brief # Product brief
/forge-prd # PRD completo
/forge-architecture # Architettura + ADR
/forge-sprint # Sprint planning
/forge-story # Prepara story
/forge-implement # Implementa story
/forge-review # Dual review (AI + Human)
/forge-retro # Retrospettiva
```

**Output:** Brief, PRD, Architecture, Epic, Stories, Sprint Status, ADR

## Track: Product

**Quando:** Nuovo prodotto, greenfield, 4+ settimane

```
/forge-init # Setup + Constitution
```

Poi segue il workflow Epic con l'aggiunta di:

- **Constitution** - Governance document immutabile
- **UX Phase** - `/forge-ux` per prodotti con interfaccia utente

**Output completo:** Tutti i documenti del track Epic + Constitution

# Gli 8 Subagent Specializzati

Agent	Modello	Ruolo
forge-analyst	Sonnet 4.6	Esplorazione, research, scope detection
forge-pm	Opus 4.7	Requirements, spec, PRD, user stories
forge-ux	Sonnet 4.6	User journeys, wireframe, accessibilità, design spec
forge-architect	Opus 4.7	Architettura, ADR, planning tecnico
forge-scrum	Sonnet 4.6	Sprint planning, story management
forge-reviewer	Opus 4.7	Adversarial review — Task A (7 dimensioni)
forge-reviewer-peer	modello indipendente	Adversarial review — Task B, indipendente
forge-qa	Sonnet 4.6	Test strategy, test generation

/forge-review lancia **forge-reviewer + forge-reviewer-peer in parallelo** e sintetizza i risultati — i *consensus findings* (entrambi i modelli) hanno la massima

## Progressive Context Chain



### Ogni fase riceve contesto strutturato dalla fase precedente

- Previene decisioni inconsistenti
- Garantisce allineamento architetturale
- Accumula conoscenza nel tempo

# Constitutional Governance

La **Constitution** è il documento di governance di massima autorità

## 9 Articoli Standard

1. **Core Principles** - Principi non negoziabili
2. **Technology Stack** - Tech stack approvato
3. **Architecture Patterns** - Pattern obbligatori
4. **Quality Standards** - Soglie qualità
5. **Security** - Requisiti di sicurezza
6. **Error Handling** - Standard gestione errori
7. **Naming & Conventions** - Convenzioni naming
8. **Testing Standards** - Test richiesti
9. **Operational Requirements** - Standard operativi

# Persistent Knowledge Base

## 3 Tipologie di Artefatti

```
.forge/knowledge/
├── adr/ # Architecture Decision Records
│ ├── 001-database-choice.md
│ └── 002-auth-strategy.md
├── decision-log.md # Session-extracted decisions
└── lessons-learned.md # Post-mortem insights
```

### Caratteristiche:

- ✓ Creati automaticamente dal plugin `session-knowledge`
- ✓ Persistono tra sessioni
- ✓ Caricati come context in ogni sessione
- ✓ Prevengono errori ripetuti

# Adversarial Review

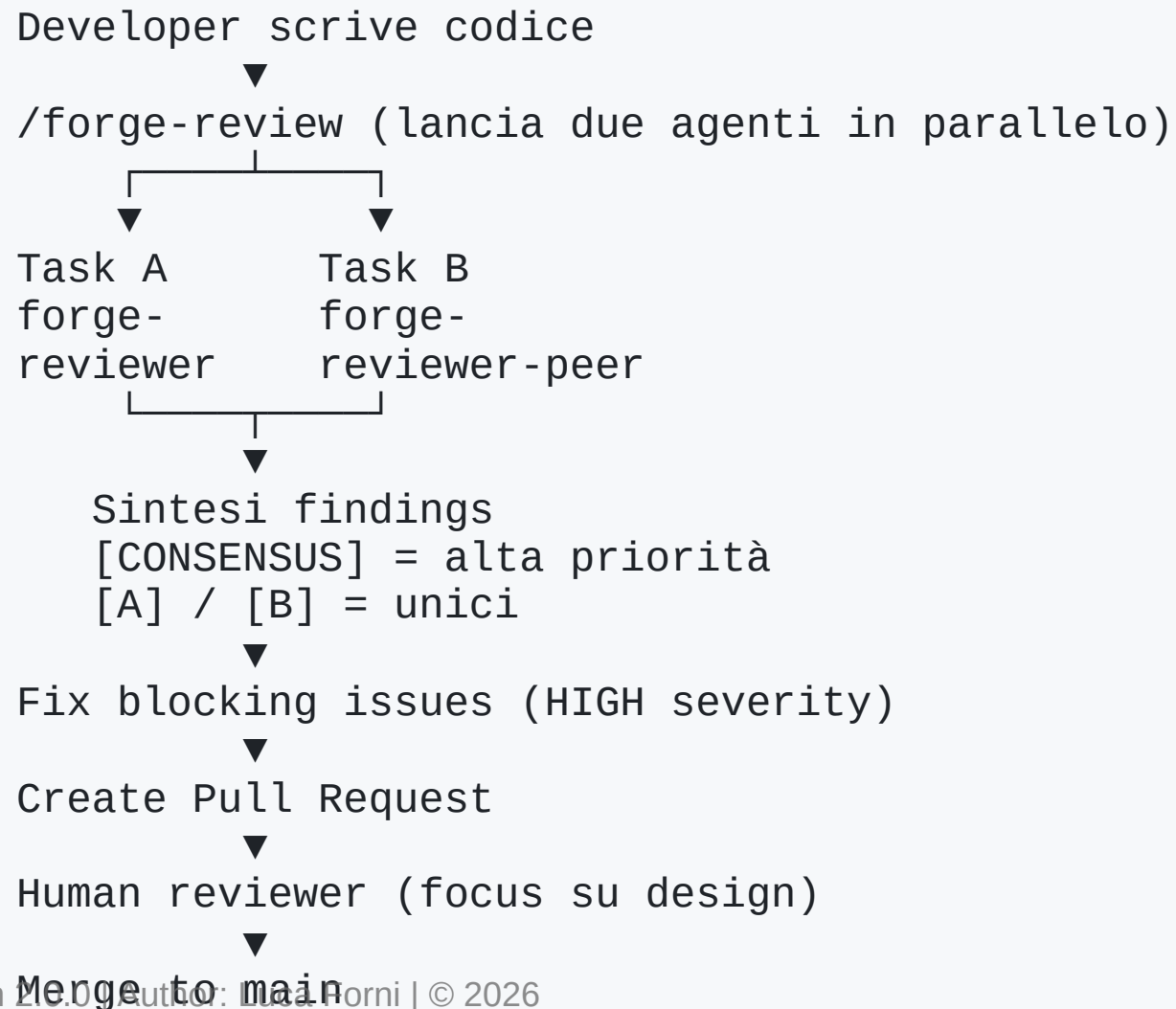
## Due Modelli in Parallelo — Review che DEVONO trovare problemi

`forge-reviewer` + `forge-reviewer-peer` esaminano su **7 dimensioni**:

1. **Correctness** - Logica, edge cases, errori
2. **Security** - Vulnerabilità, validazione input
3. **Performance** - Query inefficienti, memory leaks
4. **Maintainability** - Leggibilità, accoppiamento
5. **Constitution Compliance** - Aderenza ai principi
6. **Test-Spec Coherence** - I test coprono i requisiti della spec
7. **UX Quality** - Accessibilità (WCAG 2.1 AA), usabilità, coerenza col design

**Minimo 5 issue combined** tra i due modelli — i *consensus findings* (segnalati da entrambi) hanno la massima priorità

# Dual Review Process



## Traceability Bidirectional

```
Requirement FR-001 (spec.md)
↓
Technical approach (plan.md, section 3.2)
↓
Task 2.1 (tasks.md)
↓
Source file (src/auth/login.ts)
↓
Test file (src/auth/__tests__/login.test.ts)
```

**Custom tool:** `/forge-analyze` genera matrice di tracciabilità

- Identifica requirement non implementati
- Identifica codice senza requirement (orphan code)
- Verifica coverage test per requirement

# Brownfield Support

## Onboarding di Codebase Esistenti

```
/forge-init # In progetto esistente
```

### FORGE analizza:

- Struttura del progetto e linguaggi
- Framework e pattern architetturali rilevati
- Convenzioni di naming e organizzazione
- Tech debt potenziale

### Output:

- Constitution generata automaticamente basata sul codice esistente
- **AGENTS.md** derivato da `.eslintrc`, `tsconfig.json`, pattern rilevati

# 13 Skills Dinamiche

Skill	Utilizzata da	Scopo
adversarial-review	forge-reviewer	Protocollo review obbligatorio
advanced-elicitation	pm, architect	Tecniche di analisi profonda
scope-detection	Forge orchestrator	Valuta complessità e raccomanda track
test-strategy	forge-qa, Build	Strategia test adattiva al track
brownfield-analysis	forge-analyst	Analisi codebase esistenti
constitution-compliance	architect, reviewer	Verifica compliance alla constitution
context-chain	Tutti gli agent	Carica documenti upstream corretti
ux-design	forge-ux	Genera user journey, wireframe, a11y spec
ux-review	forge-reviewer	7ª dimensione review: qualità UX e accessibilità
frontend-pattern-library	forge-ux, Build	17 pattern UI (React + shadcn/ui)
data-presentation	forge-ux	Dashboard, visualizzazioni, data storytelling

## Plugin Automatici (1/2)

### session-knowledge

- Estrae decisioni e lezioni automaticamente
- Appende a `decision-log.md` e `lessons-learned.md`
- Inietta knowledge durante context compaction

### pre-commit-gate

- Valida consistency spec-code prima del commit
- Verifica task completati e test esistenti
- Advisory (non blocca, ma avvisa)

## Plugin Automatici (2/2)

### spec-watcher

- Monitora modifiche a `.forge/specs/`
- Rileva inconsistenze con plan/tasks
- Suggerisce `/forge-analyze`

**I plugin automatizzano governance e knowledge management senza intervento manuale**

# Benefici per Sviluppatori

Beneficio	Come FORGE lo Garantisce
Onboarding veloce	Spec, plan, ADR forniscono contesto completo
Qualità al primo tentativo	Contesto strutturato → decisioni migliori dell'AI
Meno rework dopo review	AI review cattura issue prima della human review
Requisiti chiari	/forge-clarify evidenzia ambiguità prima di implementare
Fiducia nelle decisioni	Constitution e ADR validano scelte

## Benefici per Team

Beneficio	Come FORGE lo Garantisce
Codice consistente	Constitution impone pattern; tutti gli agent seguono stesse regole
Riduce architecture drift	ADR prevengono decisioni contraddittorie
Sprint management efficiente	<code>/forge-sprint</code> e <code>/forge-status</code> automatizzano overhead
Retrospettive efficaci	<code>/forge-retro</code> produce insight azionabili
Review veloci	AI gestisce check meccanici, human focus su design
Onboarding fluido	Nuovi membri leggono <code>.forge/</code> per capire intero progetto
Parallel development sicuro	Architecture definisce boundaries; ADR prevengono conflitti

## Benefici per Enterprise

Beneficio	Come FORGE lo Garantisce
Audit trail compliance	Constitution + ADR + decision log = rationale completo
Risk management	Spec includono sezioni risk; adversarial review trova issue early
Knowledge retention	Quando sviluppatori vanno via, decisioni restano
Process standardization	Tutti i team usano stessi workflow, template, protocolli
Tech debt visibility	Brownfield analysis + traceability espongono gap
Quality metrics	Velocity sprint, issue count, spec completeness
Governance senza bottleneck	Constitution + skill impongono standard automaticamente

# FORGE vs Alternative

## vs No Methodology ("Vibe Coding")

	No Methodology	FORGE
Speed iniziale	Molto veloce	Moderata (overhead planning)
Speed nel tempo	Rallenta (debt accumula)	Sostenuta (knowledge composta)
Consistency	Random	Enforced
Rework	Alto (30-50%)	Basso (< 15%)
Onboarding	Settimane	Giorni

# FORGE vs BMAD vs Speckit

	BMAD	Speckit	FORGE
Agent system	Personas (1 LLM)	None	Real subagents
Tracks	3	1	5
Governance	Debole	Constitution	Constitution + ADR + KB
Knowledge persistence	None	Per-branch	Cross-session KB
Review	Adversarial	None	Dual Adversarial
Brownfield	Limitato	Limitato	Strutturato
Platform	IDE-agnostic	Agent-agnostic	OpenCode-native

# Quando Usare FORGE

## Use Case Ideali

Scenario	Track	Perché
Nuovo SaaS da zero	Product	Previene tech debt
Feature enterprise	Epic	Previene scope creep
Large codebase	Feature/Quick	Mantiene consistency
Compliance-heavy	Epic/Product	Audit trail completo
Team con turnover	Qualsiasi	Mantiene conoscenza

# Quando NON Usare FORGE

## ✗ Scenario Overkill

Scenario	Alternativa Migliore
Script one-off o utility	Scrivi codice direttamente
Progetti tutorial/learning	Focus su learning, non process
Hackathon prototype (discard after)	Velocità > struttura
Progetti < 1 settimana total lifespan	La doc sopravvive al codice
Safety-critical systems	FORGE + formal verification
Multi-team platform (50+ devs)	FORGE + program management enterprise

# Il Costo della Struttura

## Overhead Realistico

Attività	Overhead
Constitution	1-2 ore (una volta)
Spec (Feature)	20-30 min
Plan	15-20 min
Analyze	5 min
Review	10-15 min
Retro	15-20 min/sprint

**Totale per feature: ~1-1.5 ore**

## Break-Even Analysis

### Senza FORGE:

Implementation: 3 giorni

Rework dopo review: 0.5 giorni (frequente)

Rework dopo production: 1 giorno (occasionale)

Totale: 3.8 giorni

### Con FORGE:

Planning + review: 0.2 giorni

Implementation: 2.5 giorni (contesto migliore = più veloce)

Rework dopo review: 0.1 giorni (AI review cattura issue)

Rework dopo production: 0.1 giorni (raro con dual review)

Totale: 2.9 giorni

**Break-even tipicamente alla 2-3 feature del progetto**

## Effetto Compound

Session 1: Paghi cost constitution → overhead alto, benefit zero  
Session 10: Constitution previene bad decision → benefit > costi  
Session 50: Nuovo dev produttivo in ore grazie a .forge/  
Session 200: Auditor chiede rationale encryption → ADR-012 → 1 giorno vs 1 settimana

### Struttura ha costi decrescenti e ritorni composti

La domanda non è "possiamo permetterci l'overhead?" ma  
**"possiamo permetterci di NON averlo?"**

# Componenti Tecnici

## 9 Agents | 24 Commands | 13 Skills | 20 Templates

```
.opencode/ # Sorgente canonica (proiettata per piattaforma)
├── agents/ # 8 subagent specializzati + forge orchestrator
├── commands/ # 24 slash command
├── skills/ # 13 skill dinamiche
├── tools/ # 3 custom tool (via MCP server condiviso)
├── plugins/ # 3 plugin automation
└── templates/ # 20 document template
.claude/ # Proiezione Claude Code (agents, commands, hooks)
.codex/ # Proiezione Codex CLI (agents, commands)
```

**Un unico MCP server condiviso ( `validate-spec`, `trace-requirements`, `sprint-status` ) serve tutti e tre i runtime**

# Quick Start

## 1. Installazione (auto-detecta il tuo runtime)

```
Dal repo FORGE – OpenCode, Claude Code e Codex CLI rilevati automaticamente
npx tsx install-forge.ts /path/to/your/project

Oppure target esplicito
npx tsx install-forge.ts /path/to/your/project --platform=claude-code
```

## 2. Verifica

```
cd your-project
opencode
> /forge-help
```

## 3. Primo Workflow (Quick)

```
Version > 0.0.1 /forge-quick | © 2025
> /forge-quick "Add health check endpoint returning app version"
```

# Directory Structure



# Model Strategy

## Differenziazione per Cognitive Demand

Model	Quando	Agenti
Opus 4.7	Deep reasoning, decisioni architetturali, adversarial review	forge-pm, forge-architect, forge-reviewer
Sonnet 4.6	Velocità, good-enough reasoning, analysis, sprint mgmt	Forge, forge-analyst, forge-scrum, forge-qa
Modello indipendente	Adversarial review indipendente (secondo parere)	forge-reviewer-peer

Modelli assegnati centralmente in `opcode.json` — niente direttive nei frontmatter

# Adoption Path Incrementale

## Non Serve Adottare Tutto Subito

**Week 1-2:** Solo Hotfix + Quick

- Familiarizza con workflow senza cambiare processo esistente

**Week 3-4:** Feature track per una feature media

- Sperimenta ciclo completo spec-plan-implement-review

**Month 2:** Epic track per iniziativa più grande

- Aggiungi sprint management

**Month 3+:** Valuta Product track e constitution governance

# Comandi Essenziali

## Quick Reference Card

Tracks:	/forge-hotfix	Bug fix, 1 file, < 30 min
	/forge-quick	Small feature, 1-5 tasks
	/forge-specify	Medium feature (start here)
	/forge-brief	Large epic
	/forge-init	New product
Feature Flow:	specify → clarify → ux → plan → analyze → tasks → implement → review	
UX Commands:	/forge-ux	User journeys, wireframes, a11y
	/forge-wireframe	ASCII wireframe generation
Status:	/forge-status	Sprint dashboard
	/forge-help	Context-aware help
Knowledge:	/forge-adr	Create ADR
	/forge-retro	Sprint retrospective

## Caso d'Uso: OAuth Feature

```
1. Specifica
/forge-specify "Add OAuth2 auth with Google and GitHub"

2. Chiarifica ambiguità
/forge-clarify

3. UX Design
/forge-ux # login wireframe, user journey, a11y

4. Piano tecnico
/forge-plan

5. Valida consistenza
/forge-analyze

6. Task breakdown
/forge-tasks

7. Implementa
/forge-implement

8. Review adversarial (7 dimensioni, dual-model)
/forge-review
```

# Team Workflow

## Multi-Developer Feature Development

```
Developer A: Epic 1, Stories S001-S004 (Core Payments)
Developer B: Epic 2, Stories S001-S003 (Subscriptions)
Developer C: Epic 3, Stories S001-S003 (Webhooks)
```

### Artefatti condivisi (via git):

- `.forge/constitution.md` - Tutti seguono stessi principi
- `.forge/architecture/` - Decisioni tecniche consistenti
- `.forge/knowledge/adr/` - Tutti vedono tutte le decisioni
- `.forge/sprints/` - Scrum master aggiorna centralmente

### Prevenzione conflitti:

- Architecture definisce module boundaries

# Sprint Ceremony con FORGE (1/2)

## Sprint Planning

```
/forge-sprint
→ Review velocity
→ Seleziona stories da backlog
→ Assegna a developer
→ Update sprint-status.yaml
```

## Daily Standup

```
/forge-status
→ Vede stories assegnate e status
→ Flag blockers
```

## Sprint Ceremony con FORGE (2/2)

### Sprint Retrospective

```
/forge-retro
→ Calcolo velocity automatico
→ Lessons learned → knowledge base
```

# Knowledge Base Maintenance

Task	Frequenza	Responsabile
Review decision-log, promote to ADR	Settimanale	Tech Lead
Archive lessons-learned obsolete	Per sprint	Scrum Master
Review ADR staleness	Mensile	Architect
Verify constitution accuracy	Trimestrale	Tech Lead + PM

**15 minuti/settimana per manutenzione KB**

# Customization

## FORGE è Completamente Customizzabile

- **Constitution Template** - Adatta articoli al tuo dominio
- **Agent System Prompts** - Tuning per team style
- **Skills** - Aggiungi domain-specific reasoning
- **Templates** - Modifica structure documenti
- **Commands** - Crea workflow custom
- **Plugins** - Automazione specifica progetto

**Vedi:** `.opencode/docs/FORGE-CUSTOMIZATION.md`

# Integrazione CI/CD

## Quality Gate su Ogni Modifica

Gate	Cosa fa
CI matrix	Test su Node 20 / 22 / 24, merge bloccato su rosso
CodeQL	Analisi statica sicurezza (JS/TS, security-extended)
Scorecard	Posture supply-chain (incl. action pinnate a SHA)
Dependabot	Update settimanali (npm + Actions, raggruppati, auto-PR)
Branch protection	PR + code-owner review, linear history, no force-push
Pre-commit gate	Advisory: task done, test esistenti, constitution verificata

Ogni PR riceve adversarial review dual-model prima della human review

# Advanced Elicitation Techniques

## 6 Tecniche di Analisi Profonda

1. **Pre-mortem Analysis** - Immagina feature fallita, cosa è andato storto?
2. **First Principles Thinking** - Scomponi in componenti fondamentali
3. **Red Team / Blue Team** - Attaccante vs difensore
4. **Socratic Questioning** - Domande profonde su assunzioni
5. **Constraint Removal** - E se non avessimo constraint?
6. **Inversion Analysis** - Come garantire che FALLISCA?

Utilizzate da `forge-pm` e `forge-architect` per analisi spec e architettura

# Scope Detection

## Valutazione Automatica Complessità

### 7 Fattori:

- File interessati (1-2 → 50+)
- Task stimati (1 → 50+)
- Nuove dipendenze (0 → Stack decision)
- Schema changes (No → Full design)
- API surface changes
- Cross-module impact
- Necessità nuovi pattern

**Output:** JSON strutturato con track raccomandato + reasoning

# Test Strategy Adattiva

## Coverage Requirements per Track

Track	Test Richiesti
Hotfix	Regression test solo per bug
Quick	Unit test per nuovo/modificato codice
Feature	Unit + Integration test
Epic	Unit + Integration + E2E
Product	Unit + Integration + E2E + Performance benchmarks

Definita dalla **test-strategy** skill, eseguita da **forge-qa**

# Test Harness a 3 Livelli

Come FORGE testa sé stesso — stessa piramide per ogni progetto:

Livello	Cosa	Dove	Secret?
1. Unit	74 test Vitest (installer, MCP, validatori)	Ogni PR	No
2. Proiezione	Layout installazione per piattaforma (opencode/claude/codex)	Ogni PR	No
3. Smoke	CLI reali ( opencode run , codex exec ) vs gateway LLM	Settimanale + manuale	Sì (env-gated)

Gli smoke live assertano tool-call reali; senza key/CLI → skip, mai fail

# ADR (Architecture Decision Records)

## Quando Creare un ADR

- Scelta database, framework, major library
- Decisione API style (REST vs GraphQL vs gRPC)
- Pattern definitorio (event-driven vs synchronous)
- Trade-off (consistency vs availability)
- Qualsiasi decisione che qualcuno metterà in discussione dopo

## Formato

```
Context: Perché questa decisione serve?
Options: Quali alternative considerate?
Decision: Cosa scelto e perché?
Consequences: Effetti positivi, negativi, neutrali
Constitution Alignment: Quali articoli supporta?
```

# Retrospectives

## /forge-retro Output

Sprint 2 Retrospective

=====

Velocity: 32 pts (planned: 34, previous: 28)

What went well:

- Stripe integration straightforward grazie ad ADR chiari
- Task parallelism markers risparmiarono tempo

What could improve:

- Webhook testing richiede manual Stripe CLI setup (non in spec)
- OAuth PR richiede 2 giorni human review (bottleneck)

Action items:

- Aggiungi webhook test setup instructions a spec template
- Ruota PR reviewer per evitare single-person bottleneck

## Traceability Matrix Example

```
Requirement FR-001 (Login with OAuth)
 → Plan Section 3.2 (OAuth flow implementation)
 → Task 2.1 (Implement OAuth strategy)
 → src/auth/oauth-strategy.ts ✓
 → src/auth/__tests__/oauth-strategy.test.ts ✓

Requirement FR-002 (Account linking)
 → Plan Section 3.3 (Link accounts)
 → Task 3.1 (Link endpoint) → [NOT IMPLEMENTED] ⚠

Requirement NFR-001 (P95 < 200ms)
 → [NO TASK] → [NOT IMPLEMENTED] ⚠
```

Generata da **trace-requirements tool**

# Success Metrics

## Come Misurare il Successo di FORGE

Metrica	Target	Come Misurata
Rework rate	< 15%	Issue dopo PR merge / Total PR
Onboarding time	< 3 giorni	New dev to first meaningful commit
Architecture drift incidents	0	ADR violations detected
Knowledge retention	100%	Decisioni documentate / Decisioni totali
Sprint velocity	+20% after 3 sprint	Story points delivered
Code consistency	> 90%	Constitution compliance score

# Risorse e Documentazione

## Documentazione Completa in `.opencode/docs/`

- **FORGE-GUIDE.md** - Usage guide completa
- **FORGE-PHILOSOPHY.md** - Principi e benefici
- **FORGE-PROJECT-PLAN.md** - Architettura sistema
- **FORGE-CUSTOMIZATION.md** - Come customizzare
- **FORGE-DECISIONS.md** - Decision records della metodologia

## Community & Support

- GitHub: `lucaforni/forge`
- OpenCode Docs: `https://opencode.ai/docs`

# Roadmap Future

## Rilasciato in v2.0

- **Multi-piattaforma** - OpenCode + Claude Code + Codex CLI via installer a proiezione
- **Pipeline CI/CD** - CI matrix, CodeQL, Scorecard, Dependabot, branch protection
- **MCP server condiviso** - Custom tool su tutti e tre i runtime
- **Strategia harness test** - Unit + proiezione + smoke live su CLI

## Possibili Evoluzioni

- **AI Pair Programming Mode** - Forge assiste in real-time durante coding
- **Multi-repo Support** - FORGE coordination across microservices
- **Custom Domain Templates** - Pre-built constitution per fintech, healthcare, gaming
- **Integration con Project Management Tools** - Jira, Linear, Azure DevOps
- **VS Code Extension** - Accesso rapido ai comandi

# Takeaway Chiave

## Perché FORGE Cambia il Gioco

1. **Contesto strutturato** previene inconsistenze tra sessioni AI
2. **5 workflow track** adattano processo a complessità
3. **UX-first design** include user journeys, wireframe e accessibilità nel workflow
4. **Constitutional governance** garantisce quality bar uniforme
5. **Knowledge base persistente** azzera knowledge loss
6. **Adversarial review** cattura issue prima di production (7 dimensioni, dual-model)
7. **Multi-piattaforma** stessa metodologia su OpenCode, Claude Code e Codex CLI
8. **Brownfield support** onboarda codebase esistenti
9. **Team-ready** supporta 15+ developer con parallel development
10. **Qualità enforced in CI** (matrix test, CodeQL, Dependabot) + harness test a 3 livelli

# Inizia Oggi (1/2)

## 3 Step per Provare FORGE

### 1. Setup (5 minuti)

```
npx tsx forge/install-forge.ts /path/to/your/project
cd /path/to/your/project && opencode
```

### 2. Primo Task (5 minuti)

```
> /forge-quick "Your first small feature"
```

## Inizia Oggi (1/2)

### 3 Step per Provare FORGE

#### 3. Valuta Risultati

- Spec prodotto chiaro?
- Implementazione pulita?
- Test generati correttamente?

**Se sì → Procedi con Feature track**

**Se no → Contatta support per tuning**

# Domande?

---

## Contatti:

- Documentation: `.opencode/docs/`
- GitHub Issues: `lucaforni/forge`
- OpenCode Docs: `https://opencode.ai/docs`

# Grazie!

---

## FORGE

Framework for Orchestrated Requirements, Governance & Engineering

| Sviluppo software enterprise con AI, strutturato e sostenibile

**Autore:** Luca Forni

 [linkedin.com/in/lucaforni](https://linkedin.com/in/lucaforni)

 [github.com/lucaforni](https://github.com/lucaforni)

*Version 2.0.0 | MIT License | 2026*